Because text strings are basically arrays of characters, in this example an array of strings would then be effectively an array of arrays. For this to work properly we need to bring back the "*" symbol mentioned briefly earlier for pointers in the declaration of the data type as `char*` letting the Arduino compiler know that we intended for this to be an array of strings. We used a character array in this manner in our Decision Machine project code like the following:

```
char* allAnswers[] = {
"  As I see it,  ",
"      yes       ",
.
.
.
```

With all 20 answers declared in a similar fashion, this declaration creates an array of character strings to be displayed on our liquid crystal display. Remember, the Arduino compiler doesn't care if we put our strings in a block like this or put everything all on one line—it's personal choice really. So before we bother with some of the problems with enormous character arrays, let's look at using multidimensional arrays first.

Multidimensional Arrays

While we will usually only need arrays that are one-dimensional like the ones we have discussed so far in this chapter, Arduino C allows for arrays with any number of dimensions. Think of these as arrays of arrays similar to the character arrays used in Project 8, as in the following, for example:

```
int myArray[2][3];
```

This is a two-dimensional array, where `myArray[0]` is an array of three integers and `myArray[1]` also contains three integers combined in one multidimensional array. Multidimensional arrays are declared by simply adding an additional index in square brackets for each extra dimension, just like the earlier example. To determine how many elements a multidimensional array contains, we just need to multiply the dimensions of the array, so in this case `2 * 3 = 6` total elements. These elements are as follows:

```
myArray[0][0]
myArray[0][1]
myArray[0][2]
myArray[1][0]
myArray[1][1]
myArray[1][2]
```

To assign a value to an element in a two-dimensional array, we would just need to know the position or location of each dimension to a specific element, as in this example:

```
myArray[1][2] = 42;
```

This statement assigns the value 42 to the last element in our example array, remembering that arrays are 0 indexed. We can also assign values to a multidimensional array at the time we declare it, just like the one-dimensional arrays earlier.

```
int arrayTwo[2][2] = {{1, 2}, {3, 4}};
```

Unlike one-dimensional arrays, we should specify each array's dimension even if we are assigning values in the declaration, although strictly speaking, the first dimension does not have to be explicitly declared. With this new array, called `arrayTwo`, we have two arrays with two elements each for a total of